

▪ **Related academic work**

While several academic efforts address performance monitoring, they often differ significantly in scope, architecture, and practical applicability. You can check the ‘References’ section at the end of the article for corroboration of this statement.

▪ **Novelty of the current work**

This implementation is novel in that it integrates:

- Event-based full-stack architecture with React (client-side), Node.js (server-side), and MongoDB (database)
- Real-time system monitoring through InfluxDB, recording per-core CPU utilisation, memory, network usage, disk I/O, and API requests
- *Ad-hoc* push-based metric collection through UNIX utilities like *sar*, *jq*, and shell scripting
- Lightweight, open source stack enabling custom deployment and visualisation through Grafana dashboards.

To the best of our knowledge, no existing solution combines these specific components into a single, lightweight monitoring setup tailored for event-driven applications.

System design

▪ **Architecture overview**

The system consists of several layers:

- **Backend (Node.js and Express):** Manages the core logic for API routes, user authentication, event management, and performance logging.
- **Database (MongoDB):** Stores user information and event-related data, allowing for efficient querying and management.
- **Frontend (React):** Offers a dynamic interface for users to interact with events and view performance data.
- **Real-time analytics (InfluxDB**

+ Grafana): Collects system metrics and displays them in real time to monitor the performance of the server and application.

▪ **Key performance characteristics**

CPU core-wise data collection and monitoring: This approach gathers and tracks CPU core-wise usage statistics, which are pushed to InfluxDB for real-time performance monitoring. By utilising the ‘sar’ command, the system collects core-specific data like user time, system time, and idle time. The information is structured and fed to InfluxDB for storage, offering real-time insights into CPU core performance. The program operates in a loop, gathering data every second, and is intended to run in UNIX-like systems. Yet, it continuously uses system resources and must be manually stopped when no longer required.

CPU usage data collection and monitoring: This script retrieves aggregated CPU usage data and sends it to InfluxDB for real-time monitoring of system performance. It uses the ‘sar’ command to retrieve statistics for user time, system time, idle time, and other CPU statistics. The statistics are then parsed and sent to InfluxDB for storage. The script executes in a loop indefinitely, gathering and pushing data every second, giving constant insight into overall CPU performance. It is intended for use in UNIX-like systems and must be terminated manually when not required.

I/O statistics collection and monitoring: This aspect of the system is designed to capture and monitor I/O performance statistics in real time. It monitors key statistics such as transactions per second, read transactions, and write transactions, among other disk statistics, which are central to assessing system I/O performance. Data gets pushed into InfluxDB and saved under a given measurement to allow retrieval and analysis at will. This approach facilitates continuous monitoring so that potential I/O bottlenecks or system operation

inefficiencies may be discovered.

Memory usage data collection and monitoring: This module is built to gather real-time memory usage statistics, recording important statistics like free memory, available memory, used memory, and other related memory statistics. It also monitors committed memory, active and inactive memory. The information is structured and sent to InfluxDB for storage and analysis. It is used to monitor system memory usage in real time, which helps to identify potential memory inefficiencies or bottlenecks.

Network data collection and monitoring: This is a component that deals with gathering real-time network interface statistics. It measures such information as: received and transmitted packets, received and transmitted kilobytes, and utilisation of the interfaces. The data is structured and submitted to InfluxDB to be stored so that it is possible to have real-time visibility of network performance for various interfaces. This makes it possible to monitor network throughput and utilisation, and derive useful insights regarding network efficiency and possible bottlenecks.

Log data processing and aggregation: This task handles the parsing and aggregation of application logs before pushing metrics to InfluxDB. Log entries are processed using *jq* to extract message content and timestamps. Each new message is counted, and its most recent timestamp is recorded. These values are then pushed to InfluxDB as a time-series measurement. After processing, temporary data is cleaned up and the log file is cleared to prepare for the next cycle.

API log data processing: The system loads API log data by parsing key information, including endpoint, method, status, timestamp, and response time, from a JSON parsing tool. The timestamp is then converted to nanoseconds for accurate storage in InfluxDB. The parsed data is formatted